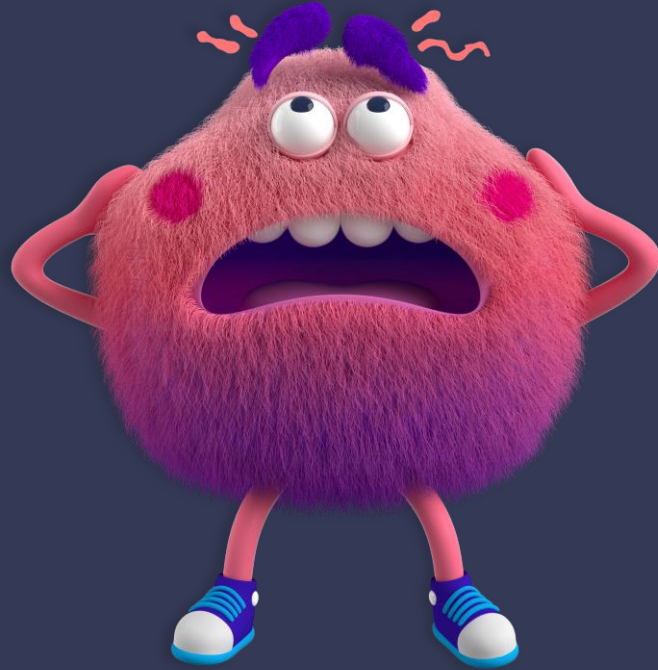
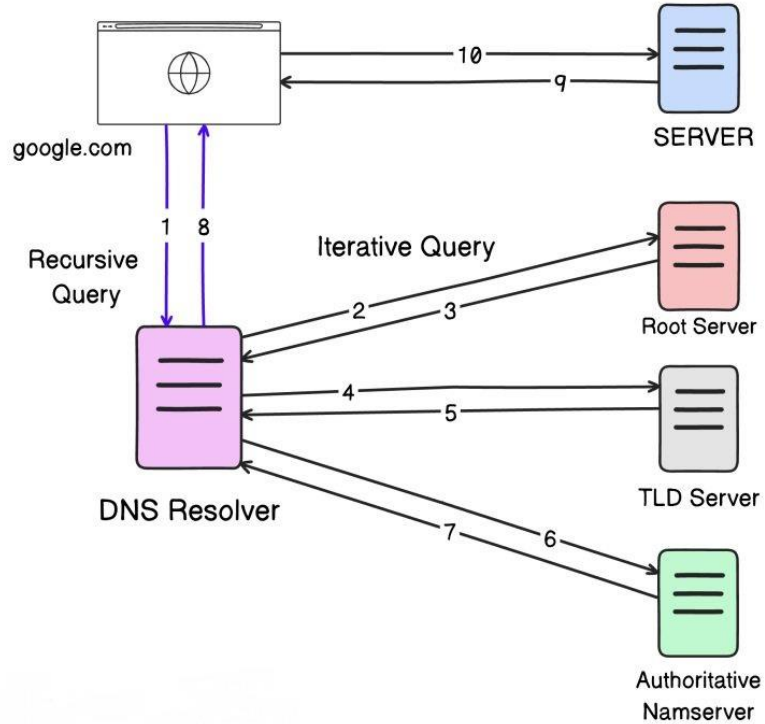


# DNS



## DNS Resolution Process



DNS



Resolution



Process

# DNS

## stands for Domain Name Systems



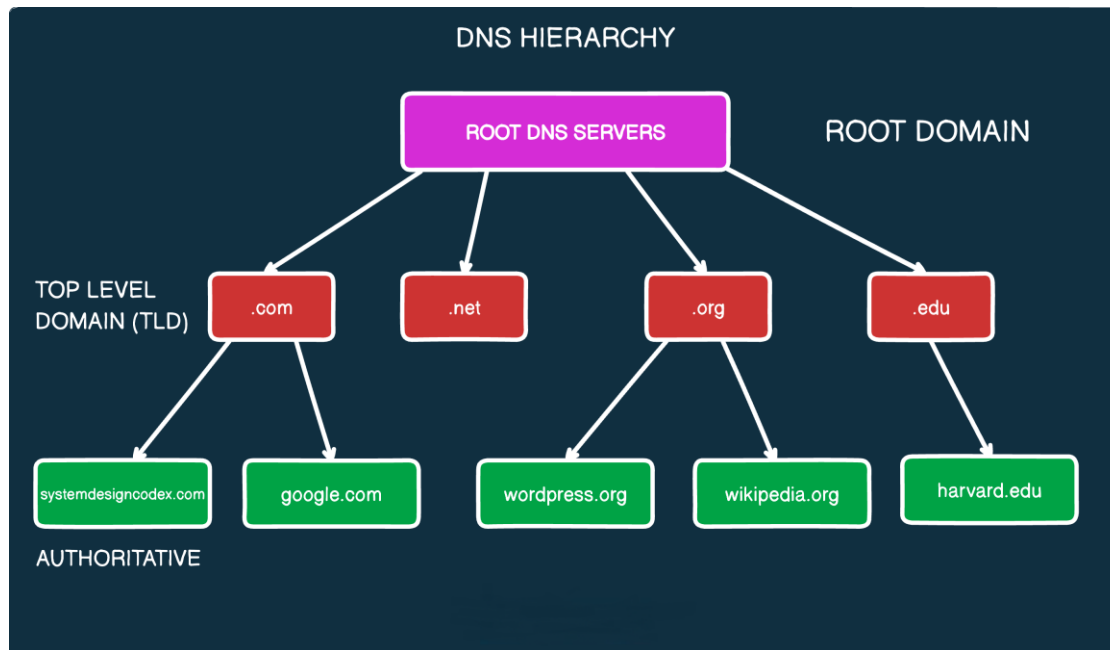
«شاید براتون سوال پیش اومده باشه که وقتی آدرس یه سایت رو تو مرورگرتون تایپ می‌کنید، دقیقاً چه اتفاقی می‌افته؟ اینجااست که پای DNS مخفف (Domain Name System) به میان می‌اد. وظیفه اصلی DNS خیلی ساده اما حیاتی: تبدیل کردن نام دامنه‌ها (مثل google.com) به آدرس‌های IP که کامپیوترها زبونش رو می‌فهمن.

به این فرآیند تبدیل اسم سایت به آدرس IP، اصطلاحاً **DNS Resolution** می‌گویند. یعنی هر بار که شما سایتی رو باز می‌کنید، این فرآیند داره خیلی سریع و بی‌سروصدا تو پس‌زمینه سیستم شما انجام می‌شه. اما تو دل این فرآیند دقیقاً چه خبره؟

برای اینکه DNS Resolution با موفقیت انجام بشه، ۴ نوع سرور DNS مختلف مثل یه تیم امدادی دست به دست هم می‌دن:

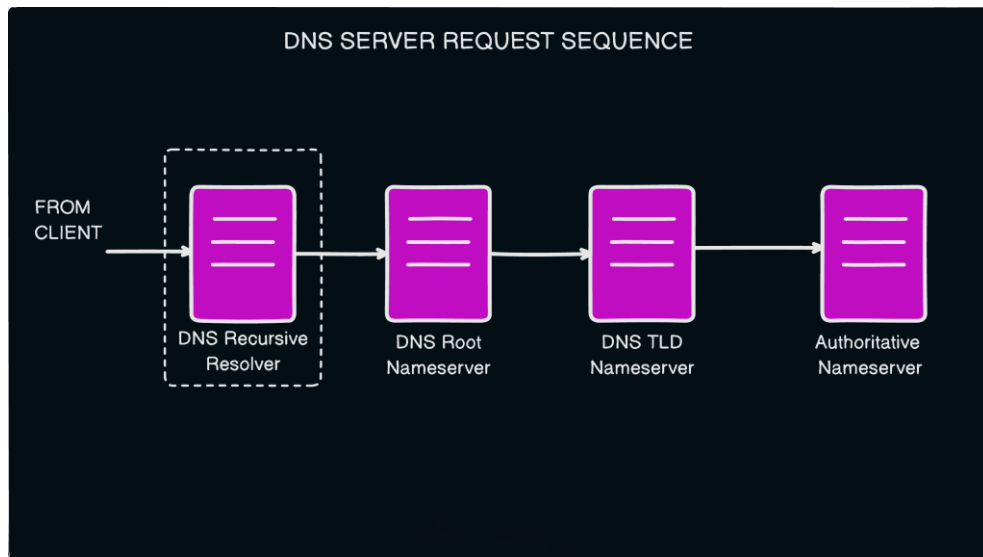
۱. **DNS Recursor** (سرور پرس‌وجوگر)
۲. **Root Nameserver** (سرور ریشه)
۳. **TLD Nameserver** (سرور دامنه سطح بالا مثل com. یا net.)
۴. **Authoritative Nameserver** (سرور معتبر یا نهایی)

اگه بخوایم ساختار سیستم نام دامنه (DNS) رو تصور کنیم، دقیقاً شبیه یه درخت وارونه کار می‌کنه. تو بالاترین نقطه این درخت، سرورهای ریشه (Root) قرار دارن. یه پله پایین‌تر، شاخه‌های TLD (مثل پسوندهای .com، .gro، و .edu) هستن و در نهایت، تو پایین‌ترین سطح این شاخه‌ها، سرورهای Authoritative (مثل [google.com](http://google.com) یا [harvard.edu](http://harvard.edu)) قرار گرفتن که آدرس IP دقیق همون سایت خاص رو تو خودشون نگه می‌دارن.»



**این ۴ قهرمان چطور کار می کنند؟**  
با نگاهی به مسیر درخواست ( DNS Server  
Request Sequence)، بیایید ببینیم هر کدام از  
این سرورها چه نقشی تو این فرآیند دارند:

# 1- DNS Recursor



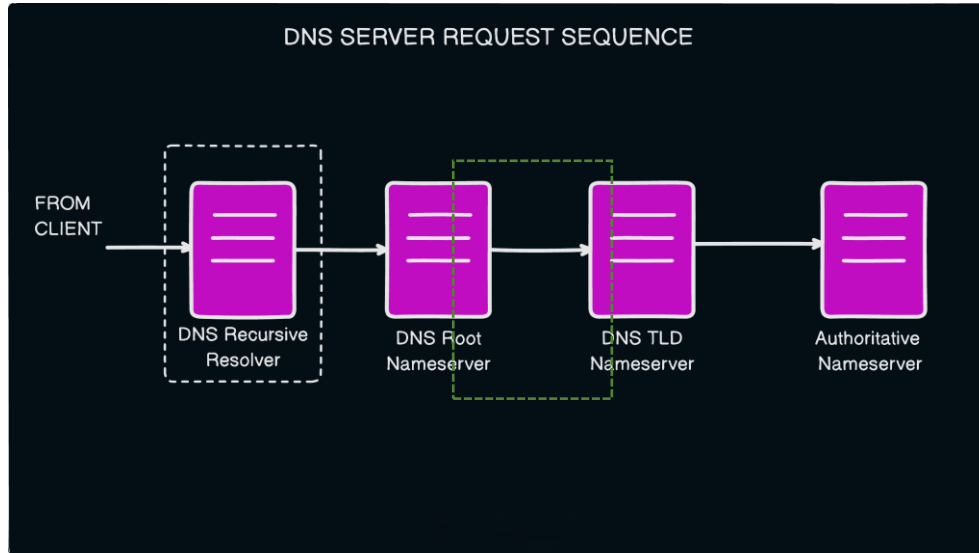
## ۱. سرور پرسشگر (DNS Recursor)؛

### کتابدار خستگی‌ناپذیر شما!

این سرور رو مثل یه کتابدار دلسوز تصور کنید. وقتی تو مرورگرتون دنبال یه سایت می‌گردید، در واقع دارید به این کتابدار می‌گید: «لطفاً این کتاب رو برام پیدا کن!». برای شما اصلاً مهم نیست کتابدار کدوم قفسه‌ها رو می‌گرده؛ شما فقط نتیجه رو می‌خواید. سرور Recursor هم دقیقاً همین‌طور عمل می‌کنه؛ از طرف سیستم شما راه می‌افته، به سرورهای مختلف سر می‌زنه و پرس‌وجو می‌کنه تا بالاخره آدرس IP رو پیدا کنه و براتون بیاره.

### ۲. سرور ریشه ( Root Nameserver)؛ نقطه شروع جستجو

اولین ایستگاهی که «کتابدار» ما بهش سر می‌زنه، سرور ریشه است. این سرور آدرس دقیق سایت رو نمی‌دونه، اما راهنمایی‌تون می‌کنه که قدم بعدی رو کجا بردارید. مثلاً می‌گه: «من آدرس دقیق رو ندارم، اما چون پسوند سایت .ir یا .com هست، باید بری پیش سرور مربوط به همون پسوند».

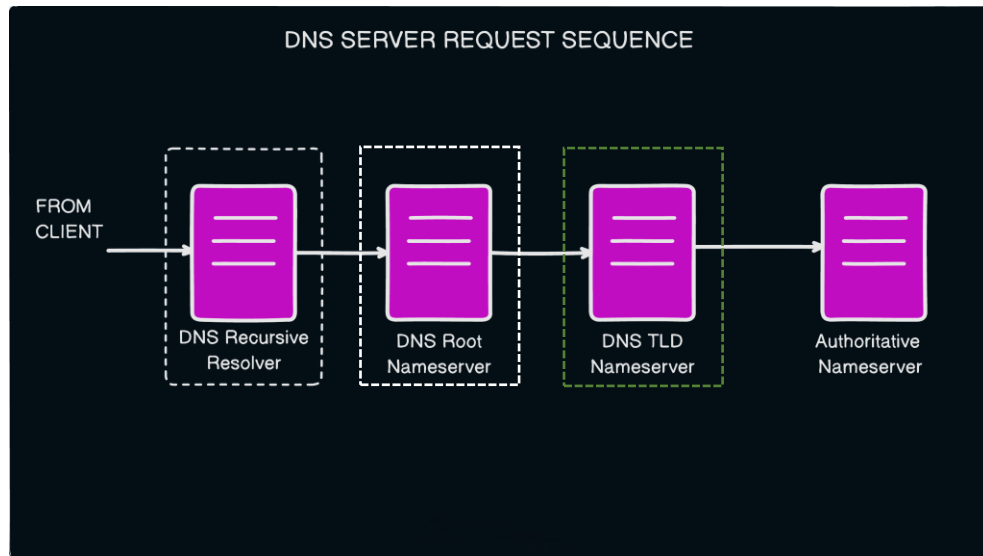




| ROOT SERVER NAME    | MANAGED BY                        |
|---------------------|-----------------------------------|
| a.root-servers.net  | Verisign                          |
| j.root-servers.net  | Verisign                          |
| b.root-servers.net  | University of Southern California |
| c.root-servers.net  | Cogent Communications             |
| d.root-servers.net  | University of Maryland            |
| e.root-servers.net  | NASA                              |
| f.root-servers.net  | Internet Systems Consortium       |
| g.root-servers.net  | US Department of Defence          |
| h.root-servers.net  | US Army                           |
| i.root-servers.net  | Netnod                            |
| k.root-seervers.net | RIPE NCC                          |
| l.root-servers.net  | ICANN                             |
| m.root-servers.net  | WIDE                              |

یه نکته جالب: همون طور که تو جدول می‌بینید، در بالاترین سطح، ۱۳ آدرس IP برای سرورهای ریشه (از a تا m) وجود داره که توسط سازمان‌های بزرگی مثل NASA، ICANN و Verisign مدیریت می‌شن. البته این به این معنی نیست که فقط ۱۳ تا سرور فیزیکی تو کل دنیا داریم! امروزه با استفاده از تکنولوژی مسیریابی Anycast، هزاران سرور در سراسر جهان پشتیبان این ۱۳ آدرس هستن تا بار ترافیک رو تقسیم کنن. از اونجایی که این سرورها تو نوک قله‌ی DNS هستن، آدرس این ۱۳ سرور از قبل تو دل نرم‌افزارهای Resolvers ذخیره شده تا همیشه بدونن جستجو رو از کجا شروع کنن.

### 3 - TLD Nameserver



سرور TLD مخفف عبارت Top-Level Domain Server است.

IANA (وظیفه ی مدیریت سرور های TLD برعهده دارد)

**سرور دامنه سطح بالا (TLD Nameserver)؛  
راهنمای قفسه ها**

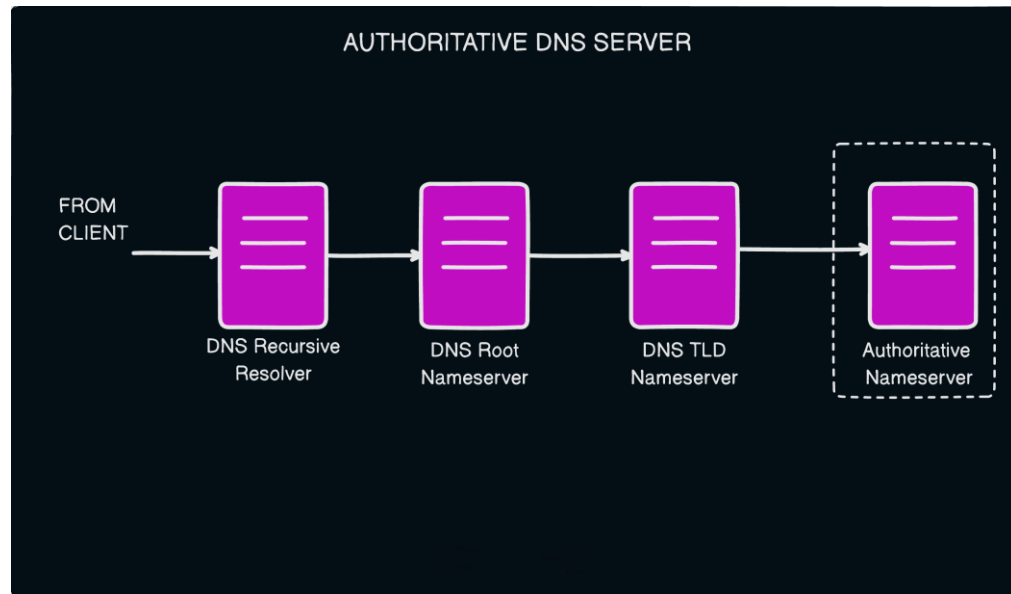
اگر سرور ریشه ما رو به یه طبقه خاص از کتابخونه راهنمایی کرده باشه، سرور TLD مسئول همون طبقه است. این سرورها (که مدیریت کلی شون با سازمان IANA است) اطلاعات تمام دامنه هایی که پسوند مشترک دارن رو نگه می دارن. سرورهای TLD به دو دسته اصلی تقسیم می شن:

✓ **دامنه های عمومی (gTLD):** مثل com، .org و .ten.

✓ **دامنه های کشوری (ccTLD):** مثل ir. برای ایران یا uk. برای انگلیس.

این سرور هم آدرس نهایی رو به ما نمی ده، بلکه ما رو به آخرین ایستگاه، یعنی سرور Authoritative هدایت می کنه.

## 4- Authoritative Nameserver

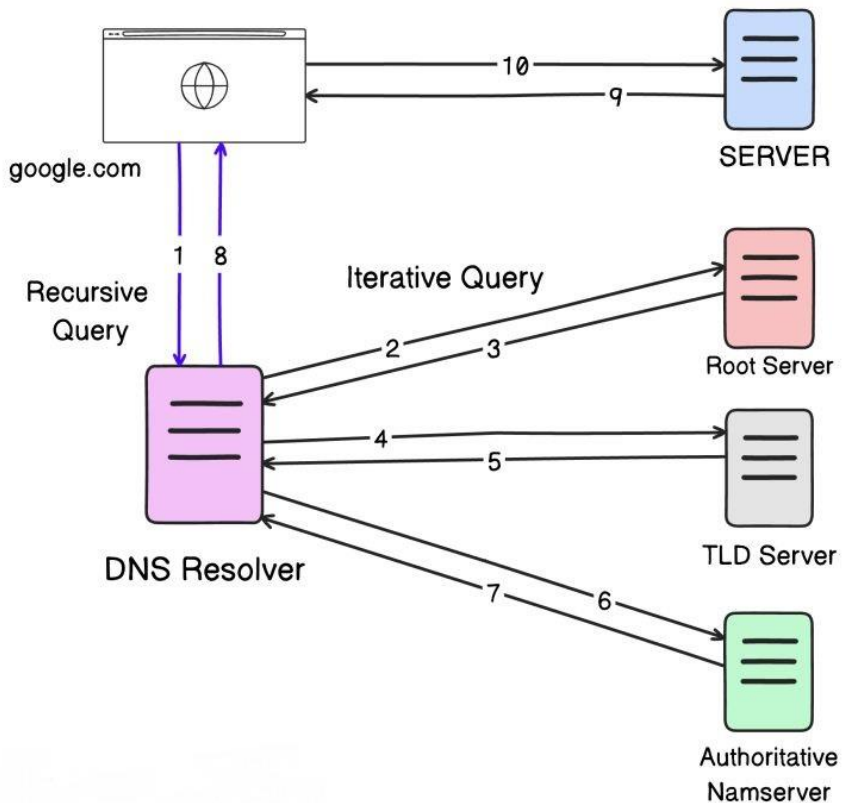


### ۴. سرور معتبر یا نهایی ( Authoritative Nameserver)؛ ایستگاه آخر

اینجا دقیقاً همون قفسه‌ایه که کتاب شما توش قرار داره! این سرور، آخرین بخش از پازل ماست و پاسخ نهایی رو تو مشت خودش داره. وقتی درخواست ما از طرف سرور TLD به این سرور می‌رسه، اگه رکوردهای مربوط به اون سایت رو داشته باشه، آدرس IP دقیق رو دودستی تقدیم سرور پرسشگر (Recursor) می‌کنه تا اونم آدرس رو به مرورگر شما برسونه و سایت باز بشه. یه مثال خیلی معروف از این نوع سرورها، سرویس‌دهنده اینترنتی «کلودفلر» (Cloudflare) هست که صاحبان سایت‌ها آدرس IP و نام دامنه‌شون رو اونجا ثبت می‌کنن تا **نقش همین سرور نهایی رو برایشون بازی کنه.**

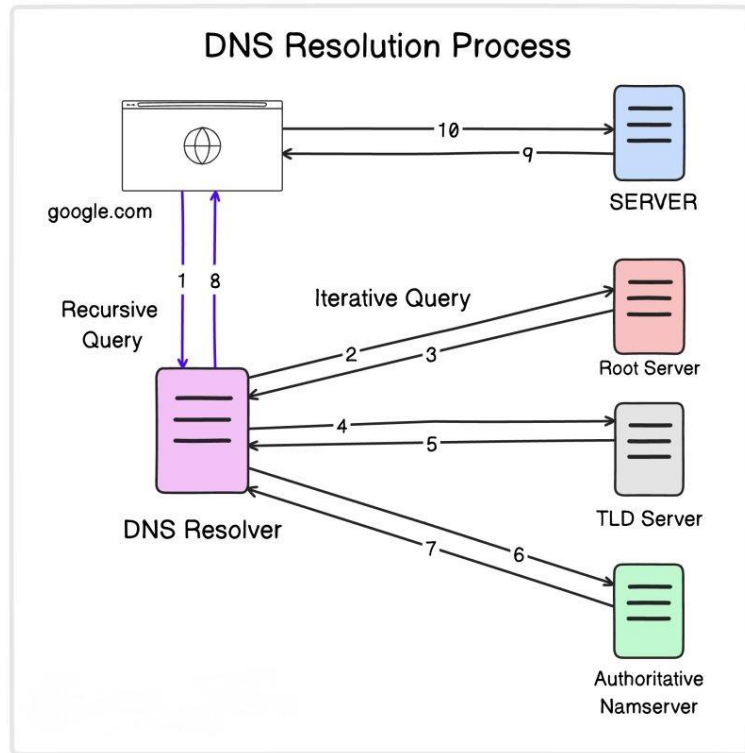
یک سفر  
واقعی: وقتی  
را google.com  
تایپ می کنید  
چه اتفاقی  
می افتد؟

## DNS Resolution Process



برای اینکه ببینیم این ۴ قهرمان چطور با هم همکاری می‌کنن،  
بیایید کل مسیر رو با بررسی فلوچارت ( DNS Resolution  
Process ) و یک مثال واقعی قدم به قدم طی کنیم. اما قبل از  
شروع حرکت، باید بدانید که ما در این مسیر ۳ نوع پرس‌وجو  
( Query ) داریم: بازگشتی (Recursive) ، تکراری (Iterative) و  
غیربازگشتی (Non-recursive).

## مرحله 1

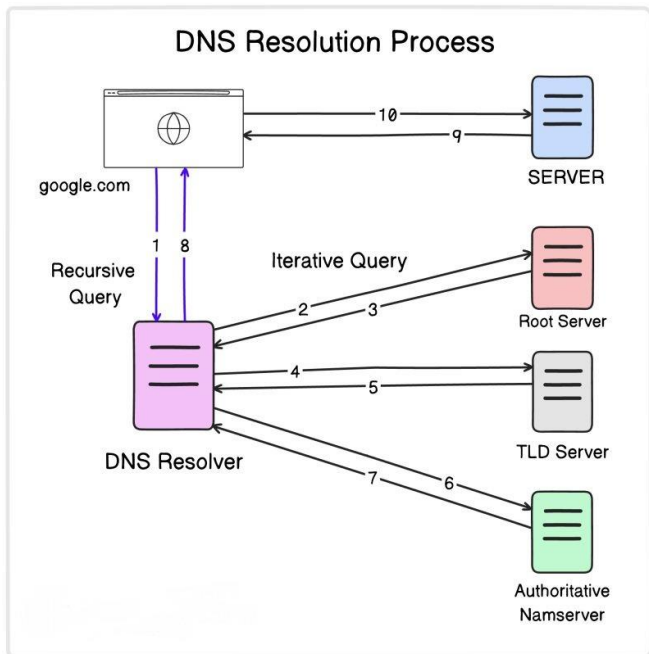


### مرحله ۱: درخواست «صفر تا صد» (پرسوجوی بازگشتی)

شما آدرس `www.google.com` رو تو مرورگر تایپ می‌کنید. مرورگر اول از سیستم‌عامل می‌پرسه: «آدرس IP این سایت رو تو حافظه (Cache) داری؟». اگه جواب منفی باشه، سیستم‌عامل یه درخواست به سرور Resolver یعنی DNS Resolver می‌فرسته. این سرور می‌تونه مال شرکت ارائه‌دهنده اینترنت شما (مثل شاتل، آسیاتک و...) باشه یا سرورهای عمومی مثل ۸/۸/۸/۸ گوگل.

به این درخواست، پرسوجوی بازگشتی (Recursive) می‌گن. یعنی سیستم‌عامل به Resolver می‌گه: «من کاری ندارم چطور پیدااش می‌کنی، یا آدرس IP رو برام بیار یا یه پیام خطا!». حالا Resolver مجبور می‌شه آستین‌ها رو بالا بزنه و سفر رو شروع کنه.

DNS Resolution Process



### مرحله ۲ و ۳: ملاقات با ریشه (پرسوجوی تکراری)

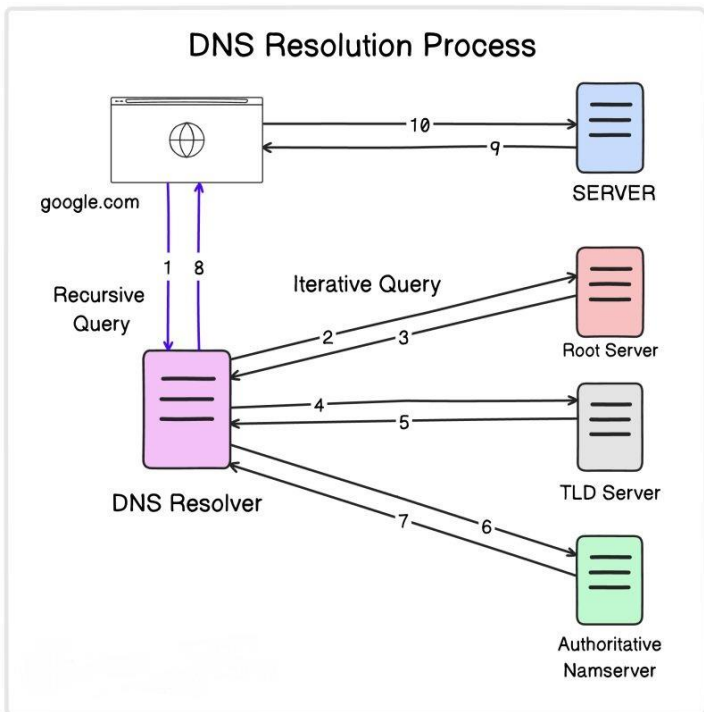
سرور resolver اول می‌ره سراغ یکی از اون ۱۳ سرور ریشه (Root Server) و می‌پرسه: «آدرس گوگل کجاست؟». سرور ریشه آدرس دقیق رو نداره، اما می‌گه: «من IP نهایی رو ندارم، اما چون سایت با .com. تموم می‌شه، آدرس سرورهای .com. رو بهت می‌دم؛ برو از اونا بپرس!».

به این نوع سوال و جواب، پرسوجوی تکراری (Iterative) می‌گن؛ چون سرور پاسخ‌دهنده، جواب نهایی رو نمیده، بلکه بهترین راهنمایی ممکن رو برای قدم بعدی ارائه می‌کنه. (همون‌طور که در تصویر زیر می‌بینید، سرور ریشه لیستی از سرورهای TLD مربوط به دامنه .com. مثل a.gtld-servers.net رو برمی‌گردونه).

|   |       |    |    |                     |
|---|-------|----|----|---------------------|
| . | 46248 | IN | NS | j.root-servers.net. |
| . | 46248 | IN | NS | h.root-servers.net. |
| . | 46248 | IN | NS | d.root-servers.net. |
| . | 46248 | IN | NS | m.root-servers.net. |
| . | 46248 | IN | NS | l.root-servers.net. |
| . | 46248 | IN | NS | f.root-servers.net. |
| . | 46248 | IN | NS | g.root-servers.net. |
| . | 46248 | IN | NS | c.root-servers.net. |
| . | 46248 | IN | NS | b.root-servers.net. |
| . | 46248 | IN | NS | i.root-servers.net. |
| . | 46248 | IN | NS | a.root-servers.net. |
| . | 46248 | IN | NS | e.root-servers.net. |
| . | 46248 | IN | NS | k.root-servers.net. |

## مرحله 4 و 5

**مرحله ۴ و ۵: پرس و جو از راهنمای قفسه‌ها (باز هم پرس و جوی تکراری)**  
 سرور resolver با لیستی که گرفته، می‌ره سراغ سرور TLD دامنه .com. و دوباره می‌پرسه: «آدرس گوگل کجاست؟». سرور TLD هم مثل ریشه می‌گه: «منم IP دقیق سرور وب گوگل رو ندارم، اما می‌دونم چه کسی مسئول دامنه [google.com](https://www.google.com) هست. برو پیش سرورهای معتبر خودِ گوگل!». اینجا هم دوباره با یه پرس و جوی تکراری طرف هستیم. (در تصویر زیر، می‌بینید که TLD آدرس سرورهای نام گوگل یعنی ns1.google.com تا ns4.google.com رو به حل‌کننده معرفی می‌کنه).

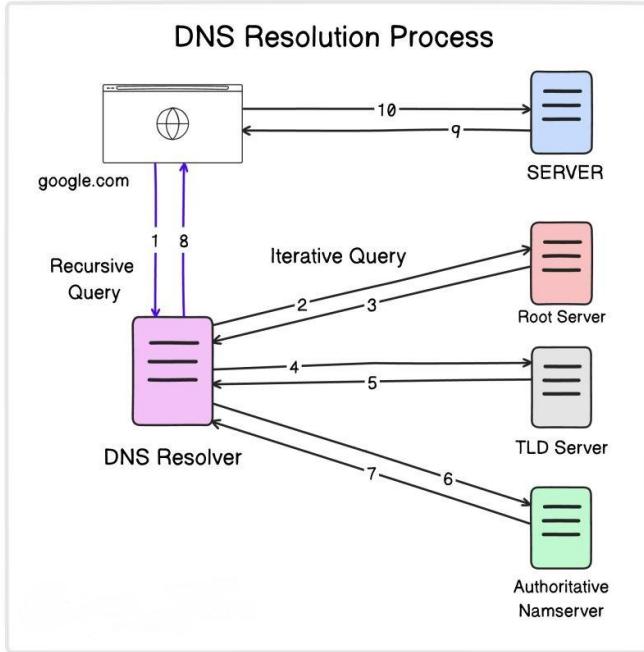


|             |        |    |    |                 |
|-------------|--------|----|----|-----------------|
| google.com. | 172800 | IN | NS | ns2.google.com. |
| google.com. | 172800 | IN | NS | ns1.google.com. |
| google.com. | 172800 | IN | NS | ns3.google.com. |
| google.com. | 172800 | IN | NS | ns4.google.com. |

پرس و جو DNS resolver از TLD نیز یک پرس و جو تکراری است.



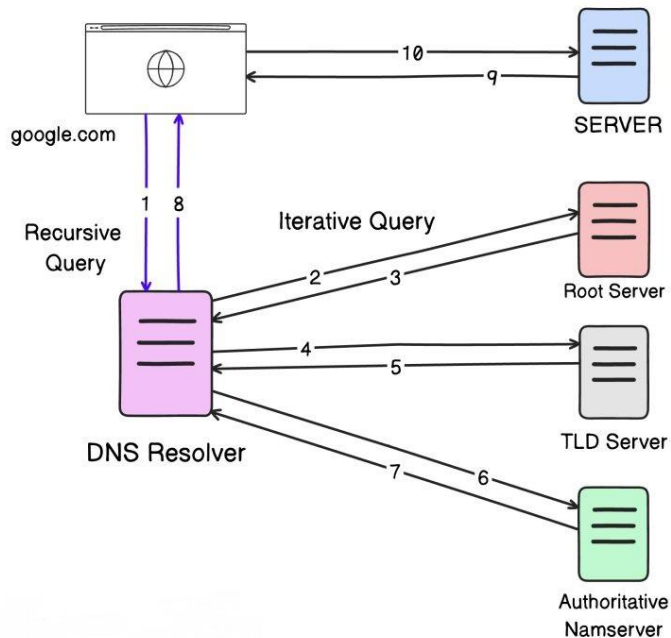
## مرحله 6، 7



### مرحله ۶ و ۷: رسیدن به ایستگاه آخر (پرس و جوی غیربازگشتی)

بالاخره سرور حل کننده می ره سراغ یکی از سرورهای معتبر گوگل (مثلاً ns1) و می پرسه: «آدرس IP سایت [www.google.com](http://www.google.com) چیه؟». از اونجایی که این سرور، رئیسِ بلامنازع این دامنه است و خودش آدرس رو در اختیار داره، بدون پاس کاری، آدرس IP دقیق رو تقدیم می کنه. به این مرحله می گن **پرس و جوی غیربازگشتی (Non-recursive)**، چون کسی که ازش سوال شده، دقیقاً همون کسیه که جواب نهایی رو تو جیبش داره.

## DNS Resolution Process



**مرحله ۸، ۹ و ۱۰: پایان ماموریت و نمایش سایت**  
 در مرحله ۸، سرور پرسشگر که با موفقیت آدرس IP رو پیدا کرده، اون رو به سیستمعامل و مرورگر شما تحویل می‌ده. حالا (در مراحل ۹ و ۱۰ فلوچارت) مرورگر شما مستقیماً با سرور اصلی گوگل ارتباط برقرار می‌کنه و سایت در کسری از ثانیه روی صفحه نمایش شما ظاهر می‌شه!

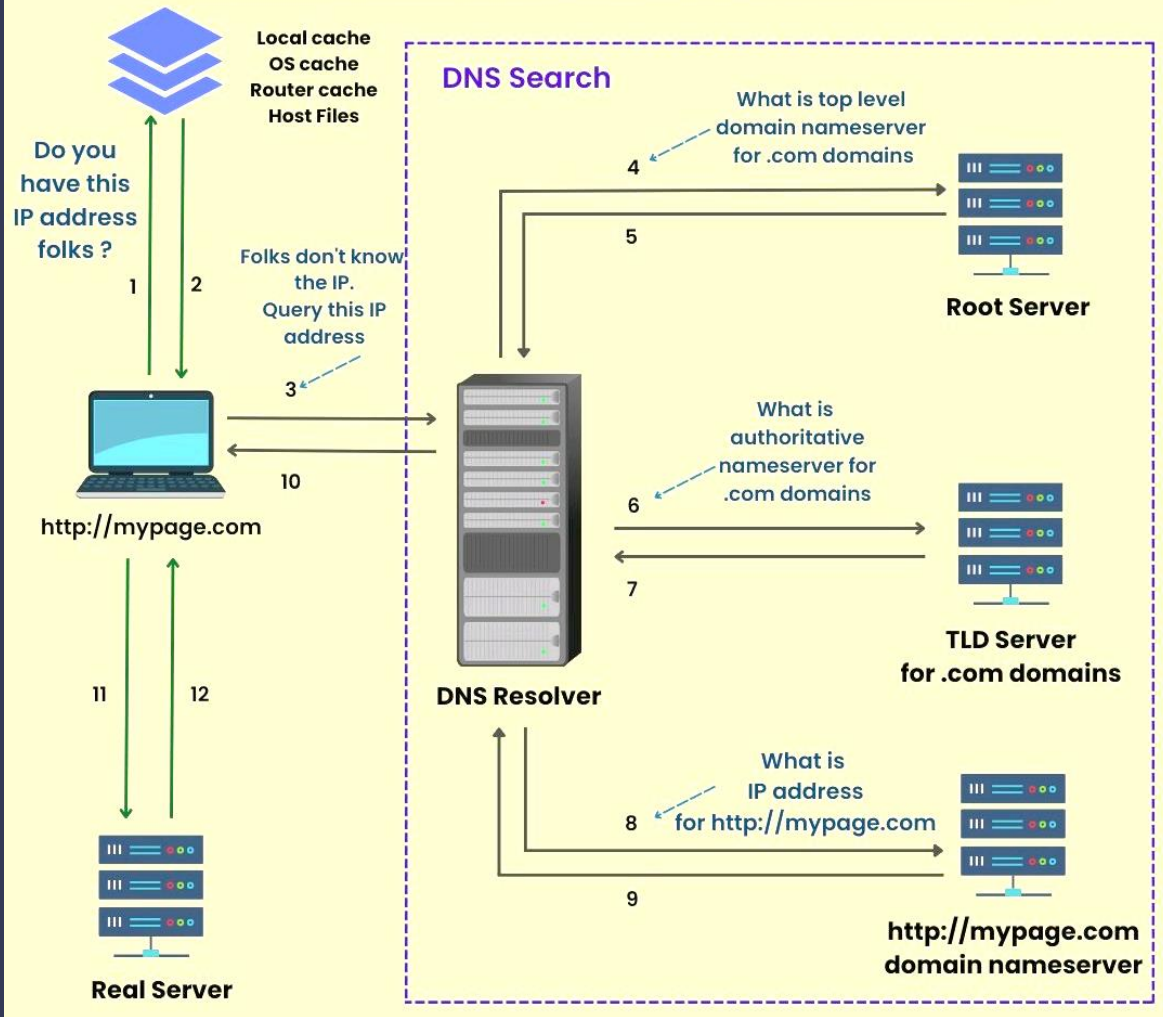
`www.google.com.`

`300`

`IN`

`A`

`142.250.194.132`



# DNS RESOLUTION PROCESS

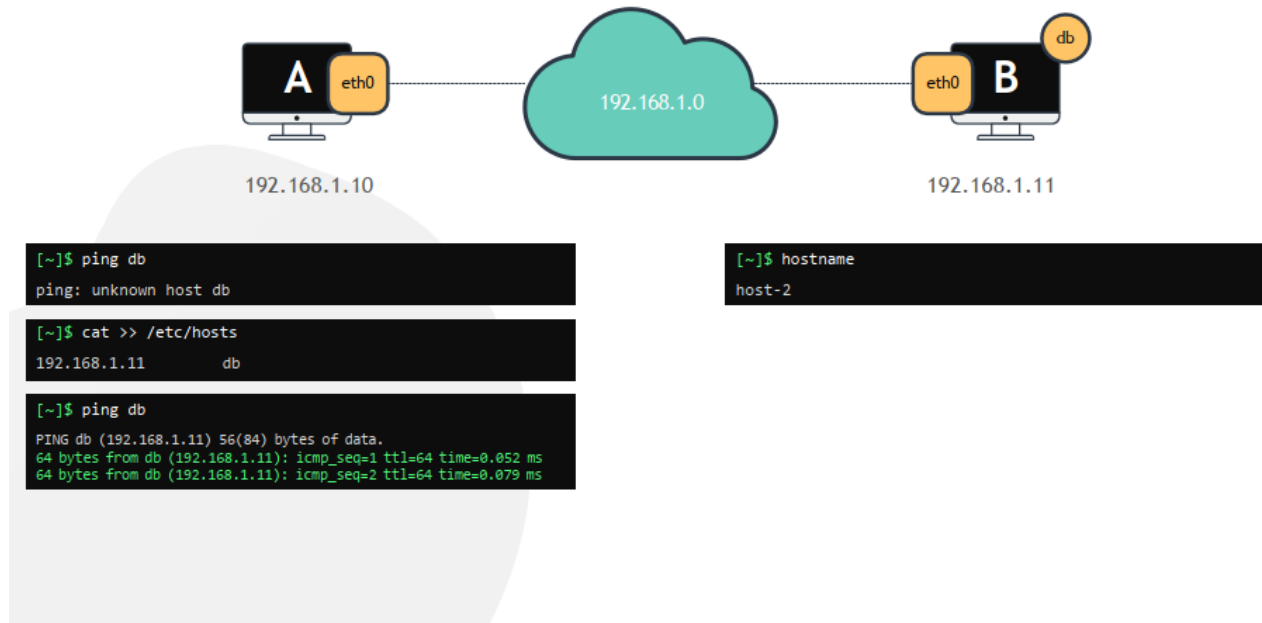


eskazemi

**بازگشت به نقطه صفر: همه چیز از سیستم عامل شروع می شود!**  
«ما معماری شگفت انگیز و جهانی DNS را بررسی کردیم و دیدیم که سرورهای Root و TLD چطور ترافیک اینترنت را هدایت می کنند. اما شاید برایتان جالب باشد که بدانید نقطه شروع این سفر پیچیده، اصلاً در اینترنت نیست؛ بلکه دقیقاً در داخل سیستم عامل خود شماست! قبل از اینکه حتی یک پکت (Packet) به سمت سرورهای جهانی ارسال شود، سیستم عامل شما ابتدا در فایل های پیکربندی محلی خودش به دنبال جواب می گردد. بیایید کمی زوم کنیم و ببینیم در داخل سیستم عامل (مثل فایل های `etc/hosts/` و `nsswitch.conf`) و شبکه های محلی چه مکانیزمی برای Name Resolution وجود دارد...»

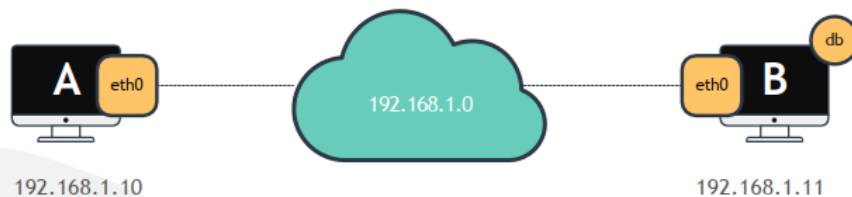
## پشت صحنه شبکه‌های محلی: وقتی همه‌چیز از etc/hosts/ شروع می‌شود!

تا اینجا دیدیم که وقتی یک آدرس اینترنتی مثل google.com را جستجو می‌کنیم، سرورهای DNS در سراسر جهان چطور با هم همکاری می‌کنند. اما بیا ببینیم به شبکه داخلی خودمان (مثلاً یک شرکت یا آزمایشگاه خانگی) برگردیم. آیا برای ارتباط بین چند سیستم ساده هم حتماً به یک سرور DNS نیاز داریم؟ پاسخ کوتاه این است: **خیر!**



همان طور که در تصاویر (ارتباط بین سیستم های A و B) می بینید، فرض کنید در شبکه محلی، سیستم A (با آدرس 192.168.1.10) می خواهد با سرور پایگاه داده یا همان B (با آدرس 192.168.1.11) ارتباط برقرار کند.

اگر در سیستم A دستور `ping 192.168.1.11` را بزنیم، همه چیز به درستی کار می کند. اما حفظ کردن IP ها برای انسان ها سخت است. دلمان می خواهد به جای IP، خیلی راحت تایپ کنیم: `ping db`. اگر بدون هیچ تنظیمی این کار را بکنیم، با خطای `unknown host db` مواجه می شویم.

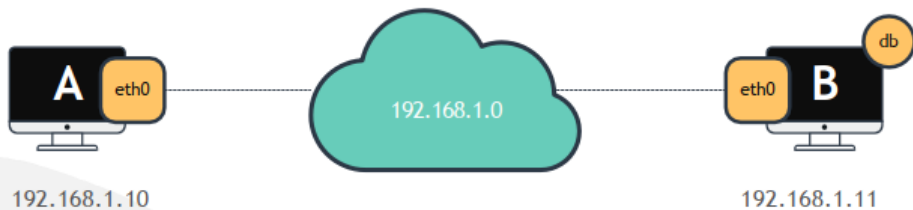


```
[~]$ ping db
ping: unknown host db
```

```
[~]$ cat >> /etc/hosts
192.168.1.11    db
```

```
[~]$ ping db
PING db (192.168.1.11) 56(84) bytes of data.
64 bytes from db (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from db (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

```
[~]$ hostname
host-2
```



```
[~]$ cat >> /etc/hosts
```

```
192.168.1.11      db
192.168.1.11      www.google.com
```

```
[~]$ ping db
```

```
PING db (192.168.1.11) 56(84) bytes of data:
64 bytes from db (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from db (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

```
[~]$ ping www.google.com
```

```
PING www.google.com (192.168.1.11) 56(84) bytes of data:
64 bytes from www.google.com (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from www.google.com (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

```
[~]$ hostname
```

```
host-2
```

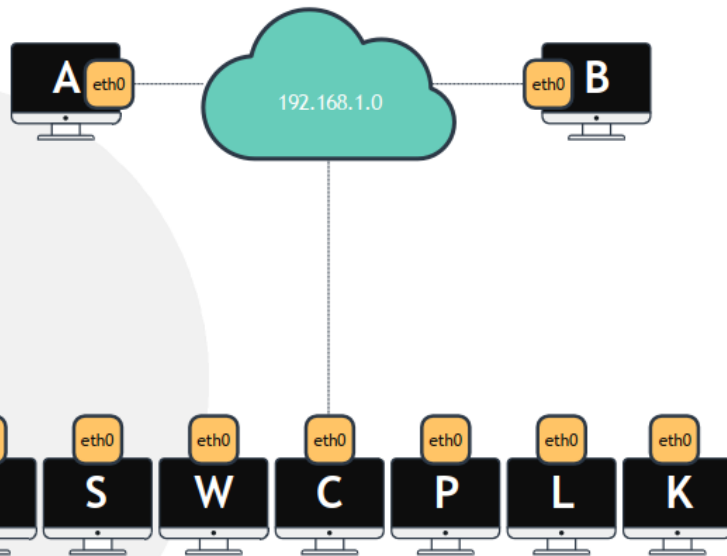
راه حل چیست؟ فایل `/etc/hosts`.  
این فایل نقش یک دفترچه تلفن محلی  
و دستی را برای سیستم عامل بازی  
می کند. کافی است در سیستم A، فایل  
`/etc/hosts` را باز کنیم و یک خط به آن  
اضافه کنیم:

db 192.168.1.11

حالا سیستم A یاد گرفته است که نام db  
معادل کدام IP است و دستور `ping db`  
با موفقیت اجرا می شود.

## Name Resolution

```
[~]$ cat >> /etc/hosts
192.168.1.10    web
192.168.1.11    db
192.168.1.12    nfs
192.168.1.20    web
192.168.1.21    db-1
192.168.1.22    nfs-1
192.168.1.30    web-1
192.168.1.31    db-2
192.168.1.32    nfs-2
192.168.1.40    web-2
192.168.1.41    sql
192.168.1.42    web-5
192.168.1.50    web-test
192.168.1.61    db-prod
192.168.1.52    nfs-4
192.168.1.60    web-3
192.168.1.61    db-test
192.168.1.62    nfs-prod
```



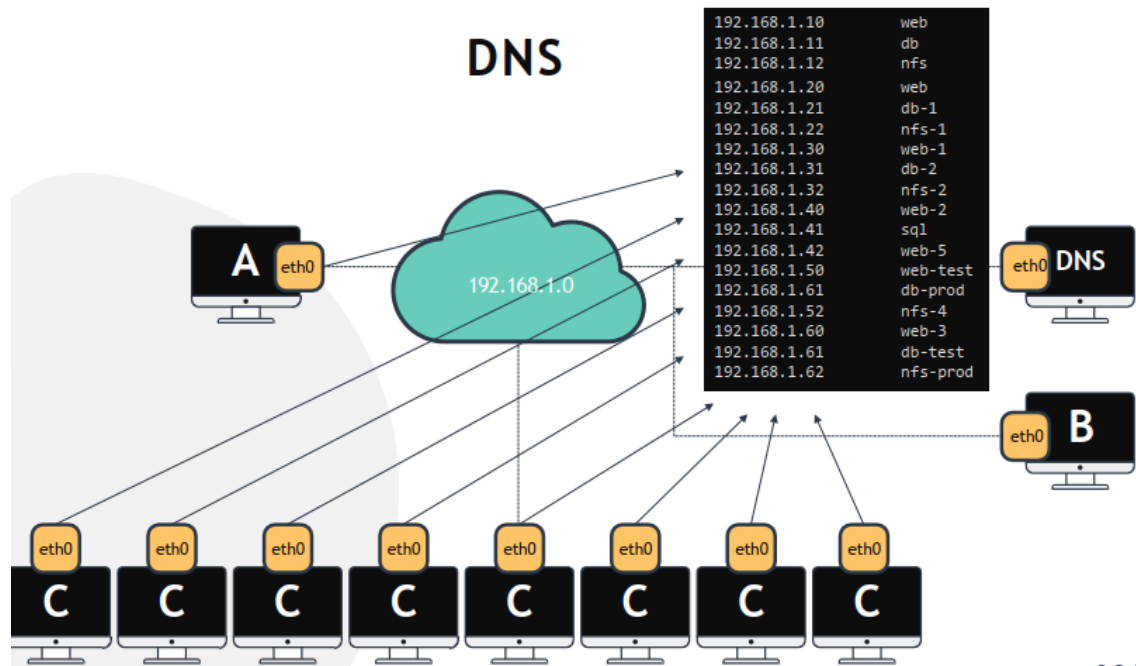
مشکل کجاست؟ چرا به DNS محلی نیاز پیدا می‌کنیم؟

استفاده از `etc/hosts/` برای ۲ یا ۳ سیستم عالی است. اما به تصویر روبرو نگاه کنید؛ جایی که تعداد سرورها (Z, D, S, W, ...) زیاد می‌شود.

اگر بخواهیم فقط از این فایل استفاده کنیم، باید لیست تمام سرورها را به صورت دستی در فایل `etc/hosts/` **تک تک کلاینت‌ها** کپی کنیم. حالا فاجعه زمانی رخ می‌دهد که IP یکی از سرورها (مثلاً سرور پایگاه داده) تغییر کند! در این حالت باید فایل کانفیگ ده‌ها سیستم را یکی‌یکی ویرایش کنیم.

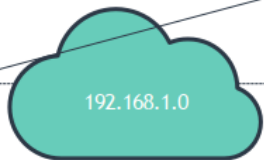
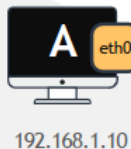


## DNS

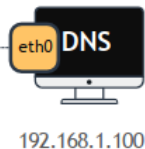


اینجاست که DNS Server محلی به داد ما می‌رسد. به جای اینکه دفترچه تلفن را در جیب همه بگذاریم، یک سرور مرکزی DNS می‌سازیم و همه رکوردها را آنجا ذخیره می‌کنیم.

## DNS



|              |          |
|--------------|----------|
| 192.168.1.10 | web      |
| 192.168.1.11 | db       |
| 192.168.1.12 | nfs      |
| 192.168.1.20 | web      |
| 192.168.1.21 | db-1     |
| 192.168.1.22 | nfs-1    |
| 192.168.1.30 | web-1    |
| 192.168.1.31 | db-2     |
| 192.168.1.32 | nfs-2    |
| 192.168.1.40 | web-2    |
| 192.168.1.41 | sql      |
| 192.168.1.42 | web-5    |
| 192.168.1.50 | web-test |
| 192.168.1.61 | db-prod  |
| 192.168.1.52 | nfs-4    |
| 192.168.1.60 | web-3    |
| 192.168.1.61 | db-test  |
| 192.168.1.62 | nfs-prod |



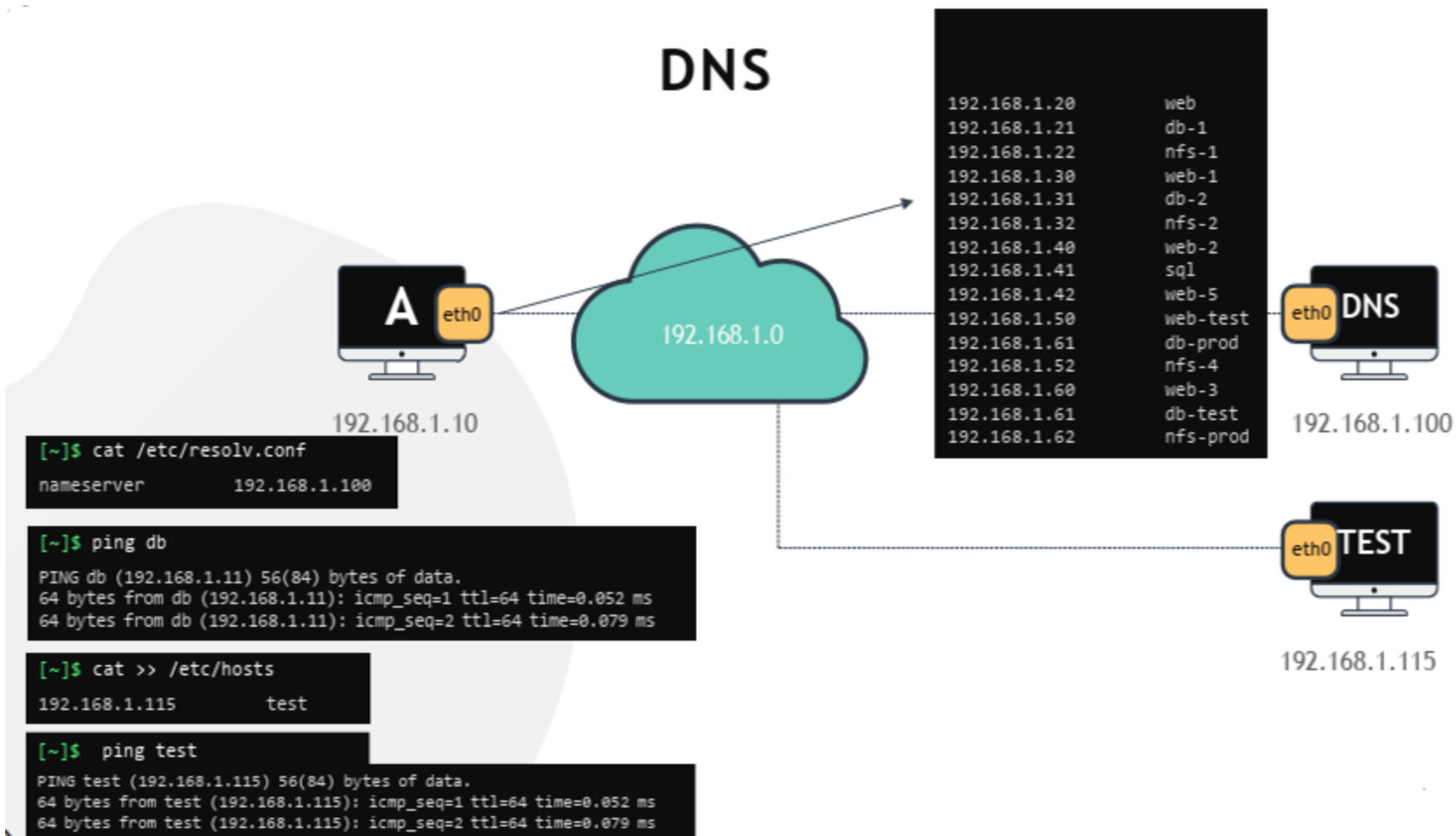
```
[~]$ cat /etc/resolv.conf
nameserver 192.168.1.100
```

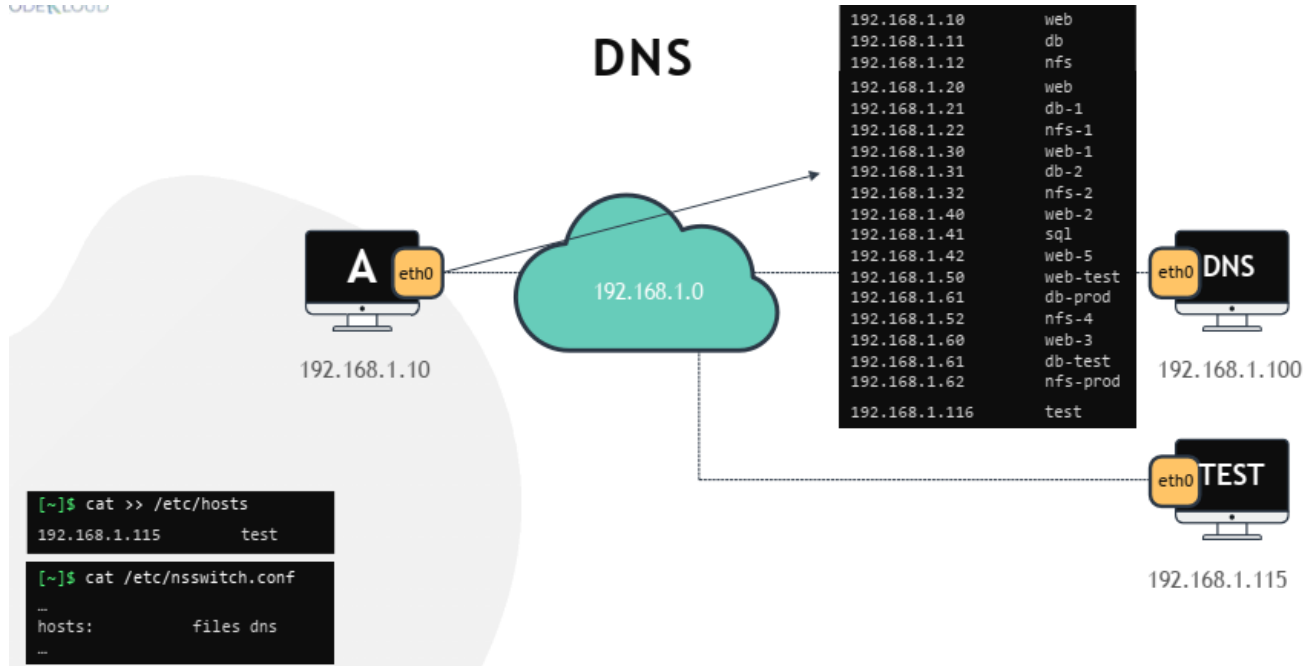
```
[~]$ ping db
PING db (192.168.1.11) 56(84) bytes of data:
64 bytes from db (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from db (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

حالا فقط کافی است به  
کلاينت ها بگوئيم: «هر وقت  
دنبال آدرسي گشتيد، از اين سرور  
بپرسيد!». اين آدرس سرور DNS  
در سيستم هاي لينوکسي، در  
فایلی به نام / etc/resolv.conf  
قرار می گیرد

مثلاً با دستور **nameserver 192.168.1.100**

# DNS





### چه کسی اولویت‌ها را تعیین می‌کند؟ فایل /etc/nsswitch.conf

حالا یک سوال مهم پیش می‌آید: اگر یک آدرس هم در فایل /etc/hosts/ سیستم باشد و هم در سرور DNS، سیستم عامل به حرف کدام یک گوش می‌دهد؟

پاسخ در دست‌فایلی به نام /etc/nsswitch.conf است. این فایل مثل یک پلیس راهنمایی و رانندگی، مسیر جستجوی سیستم را تعیین می‌کند. به صورت پیش‌فرض، تنظیمات آن به این شکل است:

hosts: files dns

این یعنی: «اول فایل‌های محلی (مثل /etc/hosts/) را بگرد، اگر پیدا نکردی، تازه برو سراغ سرور DNS!». به همین دلیل است که تغییرات /etc/hosts/ بلافاصله اعمال می‌شوند و بالاترین اولویت را دارند.

### به طور کلی وقتی دستوری مثل ping اجرا می‌شود:

1. سیستم اولویت‌ها را در `etc/nsswitch.conf/` چک می‌کند.

2. به فایل `etc/hosts/` سر می‌زند.

3. اگر جواب را نیافت، به سراغ `etc/resolv.conf/` می‌رود تا آدرس DNS سرور را بردارد و درخواست را به آن سرور (مرحله آغازین در چرخه DNS جهانی) ارسال کند.

# nslookup

```
[~]$ nslookup www.google.com
Server:      8.8.8.8
Address:     8.8.8.8#53

Non-authoritative answer:
Name:   www.google.com
Address: 172.217.0.132
```

✓ **دستورات nslookup و dig:** این ابزارها مستقیماً با سرور DNS صحبت می‌کنند و هیچ کاری با فایل `etc/hosts` ندارند! بنابراین، اگر نامی را در فایل `hosts` وارد کرده‌اید اما با `dig` آن را نمی‌بینید، تعجب نکنید؛ سیستم مشکلی ندارد، فقط این ابزارها برای تست مستقیم سرورهای DNS ساخته شده‌اند.

## DIG

### نکته طلایی برای عیب‌یابی (Troubleshooting): تفاوت ابزارها

برای تست اینکه آیا نام‌ها به درستی به IP تبدیل می‌شوند (Name Resolution)، باید ابزار درستی را انتخاب کنید:

✓ **دستور Ping:** این دستور از قوانین سیستم‌عامل پیروی می‌کند. یعنی دقیقاً ترتیب تعیین شده در `nsswitch.conf` را چک می‌کند (اول `hosts`، بعد `dns`). این دستور همیشه خوب نیست چون ممکنه در سرور مقصد پینگ بسته شده باشد.

```
[~]$ dig www.google.com
; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 28065
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;www.google.com.                IN      A

;; ANSWER SECTION:
www.google.com.                245     IN      A      64.233.177.103
www.google.com.                245     IN      A      64.233.177.105
www.google.com.                245     IN      A      64.233.177.147
www.google.com.                245     IN      A      64.233.177.106
www.google.com.                245     IN      A      64.233.177.104
www.google.com.                245     IN      A      64.233.177.99

;; Query time: 5 msec
;; SERVER: 8.8.8.8#53(8.8.8.8)
;; WHEN: Sun Mar 24 04:34:33 UTC 2019
;; MSG SIZE rcvd: 139
```

## تذکر:

ابزارهایی مثل dig و nslookup ابزارهای تخصصی برای پرس‌وجو از سرورهای DNS هستند. به همین دلیل، مکانیزم‌های محلی سیستم‌عامل (مثل فایل `etc/hosts/` و اولویت‌بندی‌های `etc/nsswitch.conf/`) را کاملاً دور می‌زنند و مستقیماً سراغ سرورهای DNS (تعریف شده در `etc/resolv.conf/`) می‌روند.

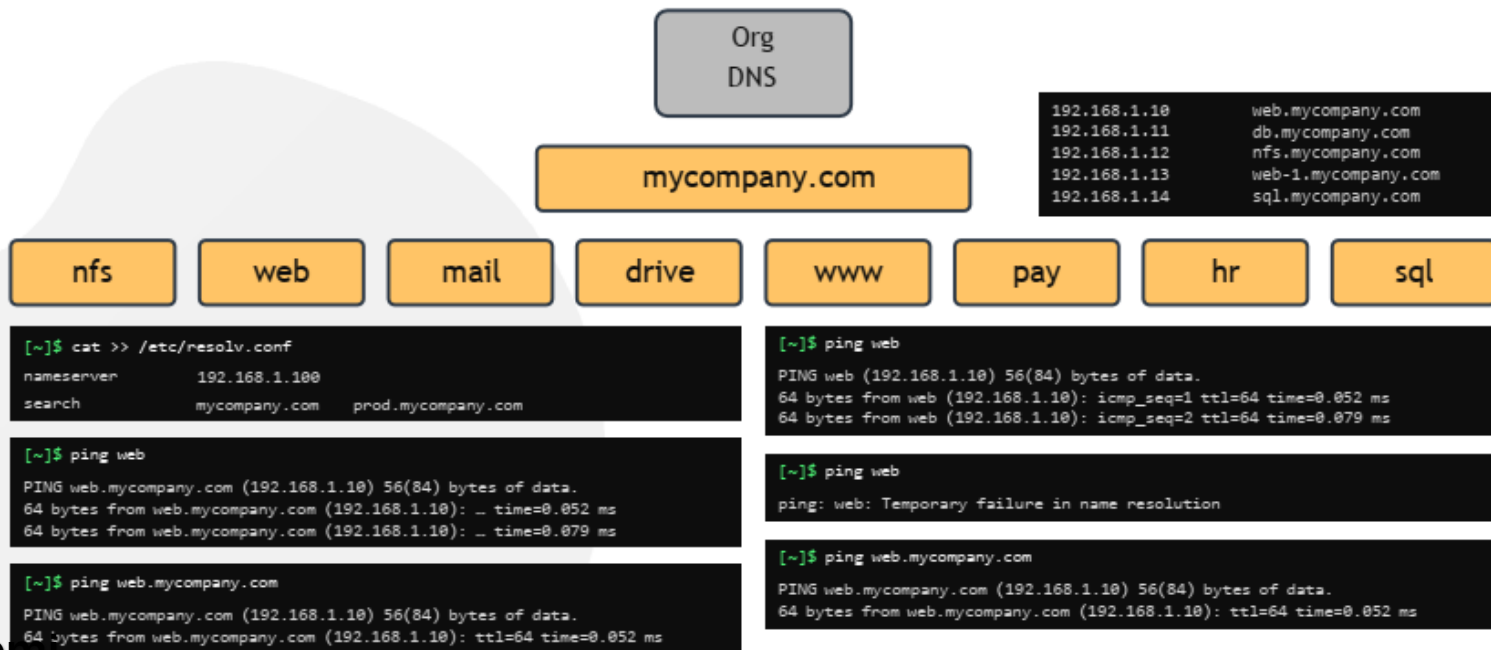
در مقابل، ابزارهای عمومی‌تر مثل ping یا مرورگرهای وب، از کتابخانه‌های استاندارد سیستم‌عامل برای پیدا کردن آدرس استفاده می‌کنند و به همین دلیل ابتدا فایل `etc/hosts/` را بررسی می‌کنند.

## داستان search domain در فایل /etc/resolv.conf

در تصویر مربوط به Search Domain، یک قابلیت به شدت کاربردی در شبکه‌های داخلی (و حتی عمومی) رو می‌بینیم.

یادتان هست گفتیم فایل /etc/resolv.conf آدرس DNS سرور را مشخص می‌کند؟ این فایل یک آپشن جذاب دیگر هم به نام search دارد.

## Search Domain





## این آپشن چه کار می‌کند؟

فرض کنید در یک شرکت بزرگ با دامین mycompany.com کار می‌کنید.  
شما ده‌ها سرور دارید مثل web.mycompany.com یا

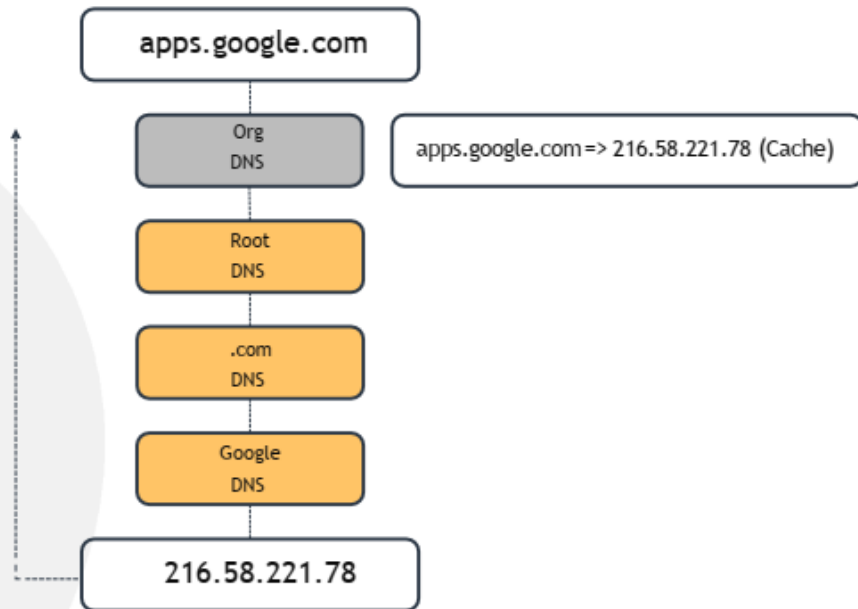
db.mycompany.com. خیلی خسته‌کننده است که هر بار بخواهید کل  
این اسم طولانی (FQDN) را تایپ کنید.

اگر در فایل resolv.conf خط زیر را اضافه کنید:

```
search mycompany.com
```

از این به بعد اگر فقط در ترمینال بزنید ping web، سیستم‌عامل شما به  
طور هوشمندانه می‌گوید: "خب این اسم کامل نیست، بذار پسوند  
mycompany.com رو بهش بچسبونم و پیرسم." در نتیجه به طور خودکار  
به دنبال web.mycompany.com می‌گردد. این کار سرعت شما را در  
شبکه داخلی به شدت بالا می‌برد.

## Domain Names



مسیر حل و فصل نام و نقش "کش" (Cache)

در تصویر Domain Names که مسیر `apps.google.com` را نشان می‌دهد، یک مفهوم حیاتی دیگر به نام **Caching (حافظه پنهان)** مطرح می‌شود.

وقتی شما `apps.google.com` را درخواست می‌کنید، DNS سرور سازمان شما (Org DNS) مجبور است سلسله مراتب را طی کند (اول از Root بپرسد، بعد از `.com` و بعد از خود Google). این کار زمان‌بر است.

**اما نکته طلایی تصویر اینجاست:** فلش کوچکی که نوشته (Cache).

وقتی Org DNS بالاخره IP این سایت (`216.58.221.78`) را پیدا کرد، آن را برای مدت مشخصی در حافظه خود (کش) نگه می‌دارد. دفعه بعد که شما یا همکاران همان سایت را بخواهید، دیگر نیازی به طی کردن آن مسیر طولانی نیست؛ Org DNS بلافاصله همان IP ذخیره شده را به شما تحویل می‌دهد و سرعت پاسخ‌گویی در حد میلی‌ثانیه می‌شود.

# Record Types

|       |                 |                                         |
|-------|-----------------|-----------------------------------------|
| A     | web-server      | 192.168.1.1                             |
| AAAA  | web-server      | 2001:0db8:85a3:0000:0000:8a2e:0370:7334 |
| CNAME | food.web-server | eat.web-server, hungry.web-server       |

## انواع رکوردهای DNS (DNS Record Types)

تصاویر مربوط به **Record Types** به ما یادآوری می‌کنند که دفترچه تلفن DNS، فقط برای پیدا کردن شماره تلفن (IP) نیست؛ بلکه اطلاعات مختلفی را نگه می‌دارد. مهم‌ترین این رکوردها عبارتند از:

• **رکوردهای A و AAAA (آدرس‌دهی):**

✓ **رکورد A:** پایه‌ای‌ترین رکورد است که یک اسم (مثل web-server) را به یک آدرس 4IPv (مثل 192.168.1.1) متصل می‌کند.

✓ **رکورد AAAA:** دقیقاً همان کار را می‌کند، اما برای آدرس‌های نسل جدید یعنی 6IPv.

✓ **رکورد CNAME (نام مستعار یا Alias):**

فرض کنید سرور وب شما اسمش food.web-server است. حالا می‌خواهید با اسم‌های eat.web-server یا hungry.web-server هم در دسترس باشد. به جای اینکه برای هر کدام یک IP جدا تعریف کنید، با CNAME می‌گویید: "هر کس eat رو خواست، بفرستش پیش همون food". (یعنی یک دامنه را به دامنه دیگر ارجاع می‌دهد، نه به یک IP).

✓ **رکورد MX (Mail Exchange):** مخصوص ایمیل است. به دنیا می‌گوید اگر کسی خواست برای دامین من ایمیل بفرستد، باید به کدام سرور

مراجعه کند.

✓ **رکورد TXT:** یک فضای آزاد برای مدیر شبکه است تا یادداشت‌های متنی بگذارد. امروزه از این رکورد به شدت برای **تایید هویت و امنیت ایمیل**

(مثل SPF و DKIM) استفاده می‌شود.

# Thanks!

**Any questions?**

**You can find me at:**



- @eskazemi



- m.esmaeilkazemi@gmail.com

